



A List of Commonly Used Git Commands

This list provides a structured overview of frequently used Git commands, categorized by their typical use in a Git workflow.

1. Configuration

Commands to set up your Git identity and preferences.

Command	Description
<code>git config --global user.name "[name]"</code>	Set the name that will be attached to your commits
<code>git config --global user.email "[email]"</code>	Set the email address for your commits
<code>git config --global init.defaultBranch main</code>	Set the default branch name for new repositories
<code>git config --list</code>	List all Git configuration settings

2. Getting & Creating Projects

Commands to start a new repository or obtain an existing one.

Command	Description
<code>git init</code>	Initialize a new, local Git repository in the current directory
<code>git clone [repository-url]</code>	Create a local copy of a remote repository

3. Basic Workflow (Staging & Committing)

Core commands for tracking changes and saving snapshots of your project.

Command	Description
<code>git status</code>	Show the status of changes (tracked, untracked, staged)
<code>git add [file-name.txt]</code>	Add a specific file to the staging area
<code>git add .</code>	Add all new and changed files in the current directory to staging
<code>git add -A</code>	Add all new and changed files in the entire working tree to staging
<code>git commit -m "[commit message]"</code>	Record staged changes to the repository with a descriptive message
<code>git commit -am "[commit message]"</code>	Stage all modified tracked files and commit them in one step

4. Branching

Commands for managing different lines of development.

Command	Description
<code>git branch</code>	List all local branches (the asterisk * denotes the current branch)
<code>git branch -a</code>	List all branches

Command	Description
	(local and remote)
<code>git branch [branch-name]</code>	Create a new local branch
<code>git checkout [branch-name]</code>	Switch to an existing branch
<code>git checkout -b [branch-name]</code>	Create a new local branch and switch to it
<code>git checkout -b [new-branch-name] origin/[remote-branch-name]</code>	Create a new local branch based on a remote branch and switch to it
<code>git branch -m [old-branch-name] [new-branch-name]</code>	Rename a local branch
<code>git branch -d [branch-name]</code>	Delete a local branch (if it has been merged)
<code>git branch -D [branch-name]</code>	Force delete a local branch (even if not merged)

Command	Description
<code>git checkout -</code>	Switch to the previously checked out branch

5. Merging & Rebasing

Commands to integrate changes from different branches.

Command	Description
<code>git merge [branch-to-merge-in]</code>	Merge the specified branch's history into the current branch
<code>git rebase [base-branch]</code>	Re-apply commits from the current branch onto the tip of <code>[base-branch]</code>
<code>git rebase -i [base-branch]</code>	Interactively rebase commits from the current branch onto <code>[base-branch]</code>
<code>git merge --abort</code>	Abort a merge operation that resulted in conflicts
<code>git rebase --abort</code>	Abort a rebase operation
<code>git rebase --continue</code>	Continue a rebase after resolving conflicts

6. Remote Repositories (Sharing & Updating)

Commands for interacting with remote repositories.

Command	Description
<code>git remote -v</code>	List all configured remote

Command	Description
	repositories with their URLs
<code>git remote add [name] [url]</code>	Add a new remote repository (e.g., <code>git remote add origin [url]</code>)
<code>git remote show [name]</code>	Show detailed information about a remote repository
<code>git remote rename [old-name] [new-name]</code>	Rename a remote repository
<code>git remote remove [name]</code>	Remove a remote repository connection
<code>git remote set-url [name] [new-url]</code>	Change the URL of a remote repository (e.g., to switch between SSH/HTTPS)
<code>git fetch [remote-name]</code>	Download objects and refs from a remote repository (does not merge)
<code>git fetch --all</code>	Fetch from all configured remotes
<code>git pull [remote-name] [branch-name]</code>	Fetch from the specified remote and merge changes into the current branch
<code>git pull</code>	Fetch from and integrate with another repository or a local branch (usually the tracked upstream branch)
<code>git push [remote-name] [branch-name]</code>	Upload local branch commits to the remote repository branch

Command	Description
<code>git push -u [remote-name] [branch-name]</code>	Push local branch and set it to track the remote branch
<code>git push</code>	Push commits from the current local branch to its configured remote upstream
<code>git push [remote-name] --delete [branch-name]</code>	Delete a branch on the remote repository
<code>git push [remote-name] :[branch-name]</code>	Alternative syntax to delete a remote branch

7. Inspecting History & Changes

Commands to view project history, changes, and differences.

Command	Description
<code>git log</code>	Show the commit history for the current branch
<code>git log --oneline</code>	Show commit history in a condensed, one-line format
<code>git log --graph --oneline --decorate --all</code>	Show a visual representation of the commit history across all branches
<code>git log --summary</code>	Show commit history with a summary of changes (files created/deleted/mode changed)
<code>git log -p [file-name.txt]</code>	Show changes over time for a specific file (patch view)
<code>git show [commit-hash]</code>	Show metadata and content

Command	Description
	changes of a specific commit
<code>git diff</code>	Show changes between the working directory and the staging area (unstaged changes)
<code>git diff --staged</code> (or <code>--cached</code>)	Show changes between the staging area and the last commit (staged changes)
<code>git diff [branch1]..[branch2]</code>	Show changes between the tips of two branches
<code>git diff [commit1]..[commit2]</code>	Show changes between two arbitrary commits
<code>git blame [file-name.txt]</code>	Show who last modified each line of a file and in which commit

8. Undoing Changes & Stashing

Commands for reverting changes, fixing mistakes, and temporarily saving work.

Command	Description
<code>git checkout -- [file-name.txt]</code>	Discard changes in the working directory for a specific file (restores last committed version)
<code>git reset HEAD [file-name.txt]</code>	Unstage a file, but keep its changes in the working directory
<code>git reset [commit-hash]</code>	Reset current HEAD to the specified commit, discarding commits after it.

Command	Description
	Staging and working directory are also reset. (Use with caution)
<code>git reset --soft [commit-hash]</code>	Reset current HEAD to specified commit, but leave staging area and working directory unchanged.
<code>git reset --hard [commit-hash]</code>	Reset current HEAD, staging area, and working directory to specified commit. All changes after this commit are lost. (Very Dangerous)
<code>git commit --amend -m "[new message]"</code>	Replace the last commit with the current staged changes and a new message
<code>git commit --amend --no-edit</code>	Add current staged changes to the previous commit without changing its message
<code>git revert [commit-hash]</code>	Create a new commit that undoes the changes made in a previous commit
<code>git clean -n</code>	Show which untracked files would be removed by <code>git clean</code> (dry run)
<code>git clean -fd</code>	Remove untracked files and directories from the working directory. (Use with caution)
<code>git stash</code>	Temporarily store modified, tracked files and staged changes
<code>git stash save "[message]"</code>	Stash changes with a descriptive message

Command	Description
<code>git stash list</code>	List all stashed changesets
<code>git stash pop</code>	Apply the latest stash and remove it from the stash list
<code>git stash apply [stash@{n}]</code>	Apply a specific stash (e.g., <code>stash@{0}</code>) but keep it in the stash list
<code>git stash drop [stash@{n}]</code>	Delete a specific stash from the stash list
<code>git stash clear</code>	Remove all stashed entries

9. File Management (Tracked by Git)

Commands for managing files within the Git tracking system.

Command	Description
<code>git rm [file-name.txt]</code>	Remove a file from the working directory and the staging area
<code>git rm -r [directory-name]</code>	Remove a directory and its contents from the working directory and staging area
<code>git rm --cached [file-name.txt]</code>	Remove a file from the staging area (Git tracking) but keep it in the working directory
<code>git mv [old-name] [new-name]</code>	Rename a file or directory and stage the change

10. Tagging

Commands for creating and managing release points or important commits.

Command	Description
<code>git tag</code>	List all tags in the repository
<code>git tag [tag-name]</code>	Create a lightweight tag at the current commit
<code>git tag -a [tag-name] -m "[message]"</code>	Create an annotated tag with a message
<code>git show [tag-name]</code>	Show information about a specific tag
<code>git tag -d [tag-name]</code>	Delete a local tag
<code>git push [remote-name] [tag-name]</code>	Push a specific tag to a remote repository
<code>git push [remote-name] --tags</code>	Push all local tags to a remote repository
<code>git push [remote-name] --delete [tag-name]</code>	Delete a tag from a remote repository

Remember that many commands have additional options and variations. Use `git help [command]` for more details.